Algoritmos y Estructuras de Datos

Guía Práctica 6

Invariante de Representación y Función de Abstracción

Segundo Cuatrimestre 2026



Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Ejercicio 1. Tenemos un TAD que modela las ventas minoristas de un comercio. Cada venta es individual (una unidad de un producto) y se quieren registrar todas las ventas. El TAD tiene un único observador:

Producto `ES` string
Monto `ES` $\mathbb{Z}$
Fecha `ES` $\mathbb{Z}$ – *segundos desde 1/1/1970*

`TAD` Comercio {
  `obs` ventasPorProducto : dicc$\langle$*Producto*, seq$\langle$tupla $\langle$*Fecha*, *Monto*$\rangle\rangle\rangle$
}

ventasPorProducto contiene, para cada producto, una secuencia con todas las ventas que se hicieron de ese producto. Para cada venta, se registra la fecha y el precio. Se puede considerar que todas las fechas son diferentes. Este TAD lo vamos a implementar con la siguiente estructura:

```
Módulo ComercioImpl implementa Comercio <
  var ventas: Vector<tupla<Producto, Fecha, Monto>>
  var totalPorProducto: Diccionario<Producto, Monto>
  var ultimoPrecio: Diccionario<Producto, Monto>
>
```

- **ventas** es una implementación de secuencia con todas las ventas realizadas, indicando producto, fecha y monto.
- **totalPorProducto** asocia cada producto con el dinero total obtenido por todas sus ventas.
- **ultimoPrecio** asocia cada producto con el monto de su última venta registrada.

a) Escribir en castellano, con precisión y detalle el invariante de representación y la función de abstracción.

b) Escribir ambos en el lenguaje de especificación.

Solución

a) En castellano:

- Invariante de representación:
  - Todos los montos y fechas de las tuplas de **ventas** deben ser positivos.
  - Todas los productos de las tuplas de **ventas** deben ser claves de **totalPorProducto** y **ultimoPrecio**.
  - Todas las claves de **totalPorProducto** y **ultimoPrecio** deben ser productos en alguna tupla de **ventas**.
  - El monto de cada producto en **totalPorProducto** debe ser igual a la suma de los montos de todas las tuplas de ese producto en **ventas**.
  - El monto de cada producto en **ultimoPrecio** debe ser igual al monto de la tupla con mayor fecha de ese producto en **ventas**.
- Función de abstracción:
  - Para cualquier tupla $\langle f, m\rangle$ de cualquier producto p en **ventasPorProducto**, la tupla $\langle p, f, m\rangle$ está en **ventas**.
  - Para cualquier tupla $\langle p, f, m\rangle$ en **ventas**, p es clave de **ventasPorProducto** y su secuencia asociada contiene a $\langle f, m\rangle$.

b) En lenguaje de especificación:

- `pred InvRep`(ci : ComercioImpl) {
  montosYFechasPositivos($ci.ventas.elems$) ∧
  ventasSonClaves($ci.ventas.elems, ci.totalPorProducto.elems, ci.ultimoPrecio.elems$) ∧
  clavesEnVentas$ci.ventas.elems, ci.totalPorProducto.elems, ci.ultimoPrecio.elems \wedge_L$
  totalEsSuma$ci.ventas.elems, ci.totalPorProducto.elems \wedge$
  ultimoEsUltimo$ci.ventas.elems, ci.ultimoPrecio.elems$
  }

  `pred montosYFechasPositivos`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle$⟩) {
  $(\forall t$: tupla $\langle Producto, Fecha, Monto\rangle)$ $(t \in ventas \rightarrow (t_1 > 0 \wedge t_2 > 0))$
  }

  `pred ventasSonClaves`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, total, ultimo$: dicc$\langle Producto, Monto\rangle$)
  {
  $(\forall t$: tupla $\langle Producto, Fecha, Monto\rangle)$ $(t \in ventas \rightarrow t_0 \in total \wedge t_0 \in ultimo)$
  }

  `pred clavesEnVentas`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, total, ultimo$: dicc$\langle Producto, Monto\rangle$) {
  $(\forall p : Producto)$ (
  $p \in total \vee p \in ultimo \rightarrow (\exists t$: tupla $\langle Producto, Fecha, Monto\rangle)$ $(t \in ventas \wedge t_0 = p)$
)
  }

  `pred totalEsSuma`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, total$: dicc$\langle Producto, Monto\rangle$) {
  $(\forall p : Producto)$ $(p \in total \rightarrow_L total[p] = sumaDeProducto(ventas, p))$
  }

  `aux sumaDeProducto`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, p : Producto$) : $Monto =$
  $\sum_{i=0}^{|ventas|-1}$ IfThenElse$(ventas[i]_0 = p, ventas[i]_2, 0)$

  `pred ultimoEsUltimo`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, ultimo$: dicc$\langle Producto, Monto\rangle$) {
  $(\forall p : Producto)$ $(p \in ultimo \rightarrow_L esUltimo(ventas, p, ultimo[p]))$
  }

  `pred esUltimo`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, p : Producto, m : Monto$) {
  $(\exists t$: tupla $\langle Producto, Fecha, Monto\rangle)$ (
  $t \in ventas \wedge t_0 = p \wedge t_2 = m \wedge esUltimaFechaDe(ventas, p, t_1)$
)
  }

  `pred esUltimaFechaDe`($ventas$: seq⟨tupla $\langle Producto, Fecha, Monto\rangle\rangle, p : Producto, f : Fecha$) {
  $(\forall t$: tupla $\langle Producto, Fecha, Monto\rangle)$ $(t \in ventas \wedge t_0 = p \rightarrow t_1 \leq f)$
  }

- `func Abs`($ci : ComercioImpl$) : $Comercio$ {
  $c : Comercio|$
  $(\forall t$: tupla $\langle Producto, Fecha, Monto\rangle)$ (
  $t \in ci.ventas.elems \rightarrow (t_0 \in c.ventasPorProducto \wedge_L \langle t_1, t_2\rangle \in c.ventasPorProducto[t_0])$
) ∧
  $(\forall p : Producto)$ $(p \in c.ventasPorProducto \rightarrow_L (\forall t$: tupla $\langle Fecha, Monto\rangle)$ (
  $t \in c.ventasPorProducto[p] \rightarrow \langle p, t_0, t_1\rangle \in ci.ventas.elems$
  ))
  }

Ejercicio 2. Considere la siguiente especificación de una relación uno/muchos entre alarmas y sensores de una planta industrial: un sensor puede estar asociado a muchas alarmas, y una alarma puede tener muchos sensores asociados.

```
TAD Planta {
    obs alarmas : conj⟨Alarma⟩
    obs sensores : conj⟨tupla ⟨Sensor, Alarma⟩⟩
    proc nuevaPlanta() : Planta {
        asegura { res.alarmas = ⟨⟩ ∧ res.sensores = ⟨⟩ }
    }
    proc agregarAlarma(inout p : Planta, in a : Alarma) {
        requiere { p = P_0 ∧ a ∉ p.alarmas }
        asegura { p.alarmas = P_0.alarmas ∪ ⟨a⟩ ∧ p.sensores = P_0.sensores }
    }
    proc agregarSensor(inout p : Planta, in a : Alarma, in s : Sensor) {
        requiere { p = P_0 ∧ a ∈ p.alarmas ∧ ⟨s, a⟩ ∉ p.sensores }
        asegura { p.alarmas = P_0.alarmas ∧ p.sensores = P_0.sensores ∪ ⟨⟨s, a⟩⟩ }
    }
}
```

Se decidió utilizar la siguiente estructura como representación, que permite consultar fácilmente tanto en una dirección (sensores de una alarma) como en la contraria (alarmas de un sensor).

```
Módulo PlantaImpl implementa Planta <
    var alarmas: Diccionario<Alarma, Conjunto<Sensor>>
    var sensores: Diccionario<Sensor, Conjunto<Alarma>>
>
```

a) Escribir en castellano, con precisión y detalle el invariante de representación y la función de abstracción.

b) Escribir ambos en el lenguaje de especificación.

Solución

a)
- Invariante de representación:
  - Para cada alarma en **alarmas**, todos los sensores de su conjunto asociado deben ser claves de **sensores** y deben tener a esa alarma en su conjunto asociado.
  - Para cada sensor en **sensores**, todas las alarmas de su conjunto asociado deben ser claves de **alarmas** y deben tener a ese sensor en su conjunto asociado.
  - No puede haber ningún sensor con un conjunto vacío de alarmas asociado.
- Función de abstracción:
  - Las claves de **alarmas** en la implementación es igual al conjunto **alarmas** en el TAD.
  - Por cada sensor asociado a cada alarma en **alarmas** en la implementación, la tupla correspondiente está en **sensores** en el TAD y viceversa.

b)
- `pred InvRep(`$ci : PlantaImpl$`) {`
  $conjuntosEnOtroDicc(ci.alarmas.elems, ci.sensores.elems) \land$
  $conjuntosEnOtroDicc(ci.sensores.elems, ci.alarmas.elems) \land$
  $ningunConjuntoVacio(ci.sensores.elems)$
  }
  `pred conjuntosEnOtroDicc(`$ds : \text{dicc}\langle S, \text{conj}\langle T\rangle\rangle, dt : \text{dicc}\langle T, \text{conj}\langle S\rangle\rangle$`) {`
  $(\forall s : S)\ (s \in ds \rightarrow_L (\forall t : T)\ (t \in ds[s] \rightarrow (t \in dt \land_L s \in dt[t])))$
  }
  `pred ningunConjuntoVacio(`$d : \text{dicc}\langle S, \text{conj}\langle T\rangle\rangle$`) {`
  $(\forall s : S)\ (s \in d \rightarrow_L |d[s]| > 0)$
  }

- `func Abs`($pi : PlantaImpl$) : $Planta$ {
  $p : Planta|$
  $claves(pi.alarmas.elems) = p.alarmas \land (\forall a : Alarma)\ ((\forall s : Sensor)\ ($
  $(a \in pi.alarmas.elems \land_L s \in pi.alarmas.elems[a]) \leftrightarrow \langle s, a \rangle \in p.sensores$
  ))
  }

Ejercicio 3. Dado el siguiente TAD:

Estudiante `ES` $\mathbb{Z}$
`TAD` Secundario {
  `obs` estudiantes : conj$\langle Estudiante \rangle$
  `obs` faltas : dicc$\langle Estudiante, \mathbb{Z} \rangle$
  `obs` notas : dicc$\langle Estudiante,$ seq$\langle \mathbb{Z} \rangle \rangle$
  `proc NuevoSecundario`(in es : conj$\langle Estudiante \rangle$) : $Secundario$ {
    `requiere` { $|es| > 0$ }
    `asegura` {
      $res.estudiantes = es \land$
      $claves(res.faltas) = es \land$
      $(\forall e : Estudiante)\ (e \in res.faltas \rightarrow_L res.faltas[e] = 0)$
      $claves(res.notas) = es \land$
      $(\forall e : Estudiante)\ (e \in res.notas \rightarrow_L res.notas[e] = \langle\rangle)$
    }
  }
  `proc RegistrarNota`(inout $s : Secundario$, in $e : Estudiante$, in $nota : \mathbb{Z}$) {
    `requiere` { $s = S_0 \land e \in s.estudiantes \land 0 \leq nota \leq 10$ }
    `asegura` {
      $s.estudiantes = S_0.estudiantes \land$
      $s.faltas = S_0.faltas \land$
      $s.notas = setKey(S_0.notas, e, S_0.notas[e] + +\langle nota \rangle)$
    }
  }
  `proc RegistrarFalta`(inout $s : Secundario$, in $e : Estudiante$) {
    `requiere` { $s = S_0 \land e \in s.estudiantes$ }
    `asegura` {
      $s.faltas = setKey(S_0.faltas, e, S_0.faltas[e] + 1) \land$
      $s.alumnos = S_0.alumnos \land$
      $s.notas = S_0.notas$
    }
  }
}

Se propone la siguiente estructura de representación:

Módulo SecundarioImpl **implementa** Secundario <

```
var estudiantes: Conjunto<Estudiante>
var faltas: Diccionario<Estudiante, int>
var notas: Vector<Conj<Estudiante>>
var notasPorEstudiante: Diccionario<Estudiante, Vector<int>>
```

>

Donde:

- En *estudiantes* están todos los estudiantes del colegio secundario
- En *faltas* tenemos para cada estudiante la cantidad de faltas que tiene hasta el momento
- En *notas* tenemos en la posición i-ésima a los estudiantes que tienen nota i
- En *notasPorEstudiante* tenemos para cada estudiante la cantidad de notas que tienen con valor i-ésimo

a) Escribir en castellano, con precisión y detalle el invariante de representación y la función de abstracción.

b) Escribir ambos en el lenguaje de especificación.

Solución

a)

- Invariante de representación:
  - Las claves de **faltas** deben ser iguales a las de **notasPorEstudiante**, que a su vez deben ser iguales a **estudiantes**.
  - **notas** debe tener 11 elementos. Lo mismo aplica para todos los vectores asociados a estudiantes en **notasPorEstudiante**.
  - Todos los enteros asociados a estudiantes en **faltas** deben ser no negativos. Lo mismo aplica a todos los elementos de todos los vectores en **notasPorEstudiante**.
  - Todos los estudiantes de cada posición de **notas** deben tener la nota correspondiente en **notasPorEstudiante**.
  - La vuelta de lo anterior, por cada nota que tiene un estudiante en **notasPorEstudiante**, el estudiante debe estar en el conjunto de la posición correspondiente en **notas**.
- Función de abstracción:
  - **estudiantes** es igual en la implementación y en el TAD.
  - **faltas** es igual en la implementación y en el TAD.
  - En el TAD, en **notas**, cada estudiante debe tener tantas apariciones de cada nota como indique la posición correspondiente a esa nota en **notasPorEstudiante**. No hace falta vuelta porque eso debería ser una igualdad.

b)

- `pred InvRep`($si : SecundarioImp$) {

$$
\begin{aligned}
&si.estudiantes.elems = claves(si.faltas.elems) = claves(si.notasPorEstudiante.elems) \wedge_L \\
&|si.notas.elems| = 11 \wedge \\
&(\forall e : Estudiante)\ (e \in si.estudiantes.elems \rightarrow_L |si.notasPorEstudiante.elems[e]| = 11) \wedge_L \\
&(\forall e : Estudiante)\ (\\
&\quad e \in si.faltas.elems \rightarrow_L (si.faltas.elems[e] \geq 0 \wedge \\
&\quad (\forall n : \mathbb{Z})\ (n \in si.notasPorEstudiante.elems[e] \rightarrow n \geq 0)) \\
&) \wedge \\
&tienenNotasCorrectas(si.notas.elems, si.notasPorEstudiante.elems) \wedge \\
&categorizadosPorNota(si.notas.elems, si.notasPorEstudiante.elems)
\end{aligned}
$$

  }

  `pred tienenNotasCorrectas`($notas : \mathsf{seq}\langle\mathsf{conj}\langle Estudiante\rangle\rangle, porEstudiante : \mathsf{dicc}\langle Estudiante, \mathsf{seq}\langle\mathbb{Z}\rangle\rangle$) {

$$
\begin{aligned}
&(\forall n : \mathbb{Z})\ (0 \leq n < |notas| \rightarrow_L (\forall e : Estudiante)\ (\\
&\quad e \in notas[n] \rightarrow (e \in porEstudiante \wedge_L porEstudiante[e][n] > 0) \\
&))
\end{aligned}
$$

  }

  `pred categorizadosPorNota`($notas : \mathsf{seq}\langle\mathsf{conj}\langle Estudiante\rangle\rangle, porEstudiante : \mathsf{dicc}\langle Estudiante, \mathsf{seq}\langle\mathbb{Z}\rangle\rangle$) {

$$
\begin{aligned}
&(\forall e : Estudiante)\ (e \in porEstudiante \rightarrow_L (\forall n : \mathbb{Z})\ (\\
&\quad 0 \leq n < |porEstudiante[e]| \rightarrow_L (porEstudiante[e][n] > 0 \rightarrow e \in notas[n]) \\
&))
\end{aligned}
$$

  }

- `func Abs`($si : SecundarioImpl$) : $Secundario$ {

$$
\begin{aligned}
&s : Secundario| \\
&si.estudiantes.elems = s.estudiantes \wedge si.faltas.elems = s.faltas \wedge_L \\
&(\forall e : Estudiante)\ (e \in s.notas \rightarrow_L (\forall n : \mathbb{Z})\ (
\end{aligned}
$$

$$0 \leq n < |si.notas.elems| \rightarrow_L si.notasPorEstudiante.elems[n] = \#apariciones(s.notas[e], n)$$
))
}

Ejercicio 4. Dados el siguiente TAD y su implementación

Ingrediente, Receta ES $\mathbb{Z}$
TAD Cocina {
obs recetario : dicc$\langle Receta, \text{conj}\langle Ingrediente\rangle\rangle$
obs stock : dicc$\langle Ingrediente, int\rangle$
proc iniciarActividad() : *Cocina* {
asegura { *Ambos diccionarios comienzan vacíos* }
}
proc nuevaReceta(inout c : *Cocina*, in r : *Receta*, in is : conj$\langle Ingrediente\rangle$) {
requiere { *La receta no está registrada previamente y el conjunto no es vacío* }
asegura { *Se registra la receta con los ingredientes requeridos* }
}
proc comprarIngrediente(inout c : *Cocina*, in i : *Ingrediente*) {
asegura { *Se le agrega una existencia al ingrediente comprado* }
}
}

Módulo CocinaImpl **implementa** Cocina <

```
/* Todas las recetas con los ingredientes que son necesarios. */
/* Cada ingrediente consume una sola existencia */
var ingredientesPorReceta: Diccionario<Receta, Conjunto<Ingrediente>>

/* Todos los ingredientes que se poseen. */
/* Cada ingrediente aparece tantas veces como existencias se tengan */
var existencias: Vector<Ingrediente>

/* Cola de prioridad ordenada de acuerdo */
/*  a la cantidad que se tiene de cada ingrediente */
var ingMasAbundante: ColaDePrioridadMax<Ingrediente>

/* Todas las cantidades con el conjunto */
/*  de los ingredientes que tienen esas existencias */
var abundancias: Diccionario<int, Conjunto<Ingrediente>>
```

>

a) Indique si cada una de las sentencias propuestas pertenecen al invariante de representación para la estructura propuesta

i) Dadas dos recetas distintas en **ingredientesPorReceta** sus conjuntos de ingredientes son disjuntos

ii) Todo ingrediente asociado a una receta en **ingredientesPorReceta** debe estar presente en **existencias**

iii) Todas las claves registradas en **abundancias** son mayores que cero

iv) Dados dos cantidades registradas en **abundancias** sus conjuntos de ingredientes son disjuntos

v) Todo ingrediente presente en **existencias** debe ser ingrediente de alguna receta registrada en **ingredientesPorReceta**

b) Proponga todas las sentencias en castellano que falten para incluir todas las restricciones correspondientes a **ingMasAbundante** en el módulo

c) Escribir la función de abstracción en lógica de primer orden

Solución

a) i) No pertenece. Dos recetas distintas podrían compartir ingredientes.

ii) No pertenece. Podemos tener recetas cuyos ingredientes no tienen stock.

iii) Dudoso, depende de la implementación. Para simplificar, decidimos que no pertenece y entonces puede haber una entrada para la cantidad 0.

iv) Pertenece. No tendría sentido que haya dos cantidades distintas para un mismo ingrediente.

v) No pertenece. La precondición de comprarIngrediente es True.

b)
- Por cada ingrediente en **ingredientesPorReceta** y **existencias**, el ingrediente está en **ingMasAbundante** con su cantidad de apariciones en **existencias** como prioridad.
- Por cada ingrediente en **ingMasAbundante**, su prioridad es su cantidad de apariciones en **existencias**.

c) `func Abs`$(ci : CocinaImpl) : Cocina$ {

$c : Cocina|$
$c.recetario = ci.ingredientesPorReceta.elems \land$
$(\forall i : Ingrediente)\ (i \in c.stock \leftrightarrow i \in ci.existencias.elems) \land_L$
$(\forall i : Ingrediente)\ (i \in c.stock \rightarrow_L c.stock[i] = \#apariciones(ci.existencias.elems, i))$

}

Ejercicio 5. Considere el siguiente TAD y una implementación

Persona, Mesa `ES` $\mathbb{Z}$
`TAD` Elecciones {

`obs` padron : dicc$\langle Persona, Mesa\rangle$
`obs` votaron : conj$\langle Persona\rangle$
`obs` votosPorCandidato : dicc$\langle Persona, \mathbb{Z}\rangle$
`proc iniciar`(in cs : conj$\langle Persona\rangle$, in vs : dicc$\langle Persona, Mesa\rangle$) : $Elecciones$
// Comienzan las elecciones con los candidatos de cs con 0 votos y el conjunto de votaron de vs vacío.
// Los candidatos pueden ser votantes
`proc votar`(inout e : $Elecciones$, in v : $Persona$, in m : $Mesa$, in c : $Persona$)
// El votante v está registrado en la mesa y todavía no había votado y el candidato c compite.
// Se registra el voto a favor del candidato en e.votaron y e.votosPorCandidato
`proc participación`(in e : $Elecciones$) : $\mathbb{R}$
// Se devuelve el porcentaje de votantes que ya sufragaron
`proc primerLugar`(in e : $Elecciones$) : $Persona$
// Se devuelve el candidato que va ganando. En caso de empate se devuelve cualquiera
// de los que vayan primeros
}

Módulo EleccionesImpl **implementa** Elecciones <

```
/* Todos los candidatos que compiten en la elección */
var candidatos: Conjunto<Persona>

/* Tiene cada mesa asociadas con los votantes registrados en ella */
var votantesPorMesa: Diccionario<Mesa, Conjunto<Persona>>

/* La prioridad es la cantidad de votos */
var ranking: ColaDePrioridadMax<Persona>

/* Todos los votantes que ya emitieron su voto */
```

```
  var sufragaron: Conjunto<Persona>

  /* Todos los votantes registrados en el padrón */
  var empadronados: Conjunto<Persona>
>
```

a) Indique si cada una de las sentencias propuestas pertenecen al invariante de representación para la estructura propuesta

i) Todo votante de **sufragaron** pertenece al conjunto asociado a alguna mesa de **votantesPorMesa**

ii) Todo votante que pertenece al conjunto asociado a alguna mesa de **votantesPorMesa** también pertenece a **sufragaron**

iii) Los conjuntos asociados a todas las mesas registradas en **votantesPorMesa** son disjuntos entre sí

iv) Cada elemento de **ranking** pertenece a **candidatos**

v) Para cada elemento de **ranking**, su prioridad no es negativa

vi) Los conjuntos de votantes **empadronados** y **candidatos** son disjuntos

vii) Todo votante de **empadronados** está presente en **sufragaron**

viii) La unión de todos los conjuntos de votantes asociados a cada mesa de **votantesPorMesa** es igual a **empadronados**

b) Escribir la función de abstracción en lógica de primer orden

Solución

a) i) Pertenece. Para votar deben estar registrados.

ii) No pertenece. Hasta que no voten no aparecen en **sufragaron**.

iii) Pertenece. No se puede estar registrado en más de una mesa.

iv) Pertenece. No se puede votar por alguien que no compite.

v) Pertenece. No puede haber una cantidad negativa de votos.

vi) No pertenece. Los candidatos pueden ser votantes.

vii) No pertenece. Mismo motivo que ii.

viii) Pertenece. Sino sobraría alguno en algún lado.

b) $\texttt{func Abs}(ei : EleccionesImpl) : Elecciones$ {

$e : Elecciones |$
$coincidePadron(e.padron, ei.votantesPorMesa.elems) \wedge$
$e.votaron = ei.sufragaron.elems \wedge$
$claves(e.votosPorCandidato) = claves(ei.ranking.elems) \wedge_L$
$(\forall p : Persona)\ (p \in e.votosPorCandidato \rightarrow_L e.votosPorCandidato[p] = ei.ranking.elems[p])$

}

$\texttt{pred coincidePadron}(padron : \mathsf{dicc}\langle Persona, Mesa\rangle, votantesPorMesa : \mathsf{dicc}\langle Mesa, \mathsf{conj}\langle Persona\rangle\rangle)$ {

$(\forall p : Persona)\ ((\forall m : Mesa)\ ($
$(p \in padron \wedge_L padron[p] = m) \leftrightarrow (m \in votantesPorMesa \wedge_L p \in votantesPorMesa[m])$
$))$

}

Ejercicio 6. Considere el siguiente TAD y una implementación

```
    Personaje, Clase, Arma ES ℤ
TAD Legion {
```

```
    obs escuadrones : dicc⟨Personaje, conj⟨Personaje⟩⟩
    obs clases : dicc⟨Personaje, Clase⟩
    obs equipo : dicc⟨Personaje, Arma⟩
    proc crearLegion(in ps : dicc⟨Personaje, Clase⟩) : Legion
        // Se crea la legión con los personajes recibidos. Los personajes se reciben sin armas
        // e inicialmente no tenemos escuadrones.
    proc formarEscuadron(inout l : Legion, in p : Personaje, in ps : conj⟨Personaje⟩)
        // El personaje p se convierte en líder del escuadrón creado con los personajes ps a su cargo.
        // Puede haber como máximo 10 escuadrones por restricción de sistema.
        // Cada personaje puede pertenecer como máximo a un sólo escuadrón.
    proc comprarArma(inout l : Legion, in p : Personaje, in a : Arma)
        // Se equipa el arma a al personaje p. Si ya tenía un arma, se descarta la previa
}
```

Módulo LegionImpl **implementa** Legion <

```
/* Todos los personajes que lideran un escuadrón */
var lideres: Conjunto<Personaje>

/* Tiene asociada a cada arma con los personajes que la tienen equipada */
var personajesPorArmas: Diccionario<Arma, Conjunto<Personaje>>

/* Relaciona a cada personaje con su clase */
var clasePorPersonaje: Diccionario<Personaje, Clase>

/* Todos los personajes que no pertenecen a ningún escuadrón */
var sinEscuadron: Conjunto<Personaje>

/* Todos los escuadrones formados con todos sus integrantes */
/*  incluyendo a los líderes */
var escuadrones: Vector<Conjunto<Personaje>>
```

>

a) Indique si cada una de las sentencias propuestas pertenecen al invariante de representación para la estructura propuesta

i) Los conjuntos de personajes asociados a cada arma en **personajesPorArma** son disjuntos entre sí

ii) Para cada arma de **personajesPorArma**, los personajes asociados no pertenecen a **sinEscuadron**

iii) En **clasePorPersonaje** no pueden existir dos personajes con la misma clase

iv) Los conjuntos **lideres** y **sinEscuadron** son disjuntos

v) Todas las posiciones de **escuadrones** tienen conjuntos no vacíos

vi) Para toda posición de **escuadrones**, los conjuntos son disjuntos entre sí

vii) No hay repetidos en **lideres**

viii) Todos los personajes de **sinEscuadron** son claves de **clasePorPersonaje**

ix) Todas las armas de **personajePorArmas** tienen asociado un conjunto con un solo elemento

x) Para cada posición de **escuadrones**, todos los personajes de cada conjunto no pertenecen a **sinEscuadron**

b) Escribir la función de abstracción en lógica de primer orden

Solución

a) i) Pertenece. A cada personaje se le asigna a lo sumo 1 arma.

ii) No pertenece. Pueden tener armas sin estar en un escuadrón.

iii) No pertenece. Esa restricción no está en el TAD.

iv) Pertenece. Si es lider, está en el escuadrón en el que lo es.

v) Pertenece. En particular, los líderes están seguro.

vi) Pertenece. Cada personaje puede pertenecer como máximo a un sólo escuadrón.

vii) No pertenece. Vale porque **líderes** es un conjunto.

viii) Pertenece. A todos los personajes les corresponde una clase.

ix) No pertenece. Dos personajes pueden tener la misma arma.

x) Pertenece. Sino es absurdo.

b) `func Abs`$(li : LegionImpl) : Legion$ {

$l : Legion|$
$coincidenEsc(l.escuadrones, li.escuadrones.elems, li.lideres.elems) \wedge$
$l.clases = li.clasePorPersonaje.elems \wedge$
$coincidenArmas(l.equipo, li.personajesPorArmas.elems)$

}

`pred coincidenEsc`$(ed : \mathsf{dicc}\langle Personaje, \mathsf{conj}\langle Personaje\rangle\rangle, es : \mathsf{seq}\langle\mathsf{conj}\langle Personaje\rangle\rangle, lideres : \mathsf{conj}\langle Personaje\rangle)$ {

$claves(ed) = lideres \wedge_L$
$(\forall p : Personaje)\ (p \in lideres \rightarrow_L (\exists c : \mathsf{conj}\langle Personaje\rangle))\ (c \in es \wedge p \in c \wedge ed[p] = c - \{p\})) \wedge$
$(\forall p : Personaje)\ (p \in ed \rightarrow_L p \in lideres \wedge ed[p] \cup \{p\} \in es)$

}

`pred coincidenArmas`$(equipo : \mathsf{dicc}\langle Personaje, Arma\rangle, personajesPorArmas : \mathsf{dicc}\langle Arma, \mathsf{conj}\langle Personaje\rangle\rangle)$ {

$(\forall p : Personaje)\ ((\forall a : Arma)\ ($
$(p \in equipo \wedge_L equipo[p] = a) \leftrightarrow (a \in personajePorArmas \wedge_L p \in personajePorArmas[a])$
$))$

}